

Proyecto Final — Bases de Datos

Sesión 3 · Implementación, poblamiento y consultas

Entregable obligatorio al cierre de la sesión

1. Objetivo de la sesión

En esta sesión llevarán su esquema relacional a una **base de datos funcional**: poblarán las tablas con volumen realista, implementarán las reglas de negocio como lógica del lado del servidor (triggers y procedimientos almacenados) y desarrollarán una batería de consultas SQL de complejidad creciente que demuestre el valor analítico de su modelo.

Este producto es el insumo de la Sesión 4 (optimización y preparación del pitch). Si no se completa aquí, bloquea las fases siguientes y afecta los criterios de *Consultas SQL y complejidad* (20 puntos), *Integridad y restricciones* (puntos restantes) y *Poblamiento, calidad y documentación* (10 puntos).

Prerrequisito: el script DDL de la Sesión 2 debe ejecutarse **sin errores** en una instancia limpia del SGBD. Si tienen correcciones pendientes, resuélvanlas **antes** de esta sesión. Trabajar sobre un esquema con errores estructurales invalida todo lo que construyan encima.

2. Agenda de trabajo

encab

- | | | |
|---|---|--|
| 1 | Poblamiento de datos. Insertar datos realistas y en volumen suficiente. Mínimo 1 000 filas en la tabla transaccional principal; mínimo 10 filas en cada tabla maestra. Los datos deben cubrir casos normales, casos límite y escenarios que activen las reglas de negocio. | Script .sql de inserciones ejecutable. |
| 2 | Reglas de negocio: triggers. Implementar al menos 2 triggers que apliquen restricciones no expresables con DDL estándar (ej: validar aforo, impedir transiciones de estado inválidas, registrar auditoría de cambios). | Script de triggers documentado. |
| 3 | Reglas de negocio: procedimientos y funciones. Implementar al menos 2 procedimientos almacenados y 2 funciones que encapsulen lógica de negocio relevante. Al menos uno debe usar cursores y manejo de excepciones. | Script de procedimientos y funciones. |

encab

filaz	Batería de consultas SQL. Desarrollar al menos 15 consultas de complejidad creciente, organizadas en niveles (sección 5). Cada consulta debe estar precedida de un comentario con la pregunta de negocio que responde.	Script de consultas documentado.
4		
5	Pruebas integrales. Ejecutar la secuencia completa: DDL → DML → triggers → procedimientos → consultas. Verificar que todo funcione de extremo a extremo. Probar al menos 3 violaciones intencionales para demostrar que los triggers responden correctamente.	Evidencia de ejecución exitosa.
filaz	Validación docente. Cada equipo muestra la base de datos funcionando: ejecución de consultas representativas, demostración de triggers y procedimientos.	Aprobación o lista de ajustes.
6		

3. Poblamiento de datos

3.1 Requisitos de volumen

encab		
Tabla transaccional principal	1 000	Volumen suficiente para que las consultas de agregación y las funciones de ventana produzcan resultados significativos.
filaz Tablas maestras / catálogos	10 por tabla	Variedad suficiente para probar JOINS y filtros.
Tablas asociativas (M:N)	50	Relaciones cruzadas realistas entre entidades.

3.2 Criterios de calidad de los datos

- **Realismo:** los datos deben parecer reales (nombres plausibles, fechas coherentes, montos en rangos razonables). No usar “aaa”, “test1”, “123”.
- **Cobertura:** los datos deben incluir casos que activen **todas** las reglas de negocio y que permitan probar **todas** las consultas.
- **Casos límite:** incluir al menos un caso que se acerque al límite de cada restricción (ej: aforo casi lleno, fecha en el borde del rango).
- **Generación:** pueden usar scripts auxiliares (Python, generadores de datos) para producir el volumen, pero los datos deben ser coherentes. Entregar también el script generador si lo usaron.

Regla de reproducibilidad: el script DML debe poder ejecutarse inmediatamente después del DDL en una instancia limpia y poblar toda la base de datos sin intervención manual.

4. Reglas de negocio: triggers, procedimientos y funciones

4.1 Triggers (mínimo 2)

Cada trigger debe implementar una restricción que **no puede expresarse** con CHECK, FOREIGN KEY u otra restricción declarativa. Ejemplos válidos:

- Impedir que el número de inscripciones supere el aforo máximo.
- Validar transiciones de estado (ej: un pedido “Entregado” no puede volver a “Pendiente”).
- Registrar automáticamente en una tabla de auditoría todo cambio en una tabla sensible.
- Actualizar un saldo o contador acumulado al insertar/eliminar en una tabla de detalle.

Cada trigger debe documentarse con:

1. La regla de negocio que implementa (en lenguaje natural).
2. Evento (BEFORE/AFTER), operación (INSERT/UPDATE/DELETE) y granularidad (FOR EACH ROW/sentencia).
3. Un caso de prueba positivo (operación permitida) y uno negativo (operación rechazada).

4.2 Procedimientos almacenados (mínimo 2)

Deben encapsular operaciones de negocio que involucren **múltiples tablas o pasos**. Ejemplos:

- Registrar una transacción completa (insertar en tabla maestra y detalles, actualizar inventario).
- Cerrar un periodo: calcular totales, cambiar estados masivamente, generar resumen.

4.3 Funciones (mínimo 2)

Deben retornar un valor calculado útil para consultas o reportes. Ejemplos:

- Calcular el total facturado de un cliente en un rango de fechas.
- Obtener el porcentaje de ocupación de un recurso.
- Determinar la categoría de un elemento según reglas de clasificación.

4.4 Requisito de complejidad

Al menos **un** procedimiento o función debe utilizar:

- Un **cursor explícito** para recorrer un conjunto de resultados.
- Manejo de **excepciones** (EXCEPTION/WHEN) para controlar errores previsibles.

5. Batería de consultas SQL

Deben desarrollar al menos **15 consultas** organizadas en los siguientes niveles de complejidad. Cada consulta debe estar precedida por un comentario que indique **la pregunta de negocio que responde**.

N1	Selección y filtrado	2	SELECT con WHERE compuesto, BETWEEN, LIKE, IN. Filtros sobre fechas y estados.
filaz N2	JOINS múltiples	3	Combinar al menos 3 tablas. Usar INNER JOIN, LEFT JOIN y al menos un caso donde la diferencia entre ambos sea significativa para el resultado.
N3	Agregación	3	GROUP BY con HAVING. Funciones COUNT, SUM, AVG, MIN, MAX. Al menos una consulta con agrupación sobre múltiples columnas.
filaz N4	Subconsultas	3	Al menos una subconsulta correlacionada, una con EXISTS/NOT EXISTS, y una en cláusula FROM o WITH (CTE).
N5	Funciones de ventana	2	Usar ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD o SUM() OVER() para análisis comparativo, rankings o cálculos acumulados.
filaz N6	Vistas	2	Al menos 2 vistas (CREATE VIEW) que encapsulen consultas frecuentes o complejas y simplifiquen el acceso a información de negocio.

5.1 Ejemplo de formato esperado

```
-- =====
-- N3-01: Cual es el ingreso total por categoria
--       de producto en el ultimo trimestre?
-- =====
SELECT c.nombre          AS categoria,
       COUNT(d.id_detalle) AS unidades_vendidas,
       SUM(d.cantidad * d.precio_unitario) AS ingreso_total
FROM   detalle_pedido d
       JOIN producto p ON d.id_producto = p.id_producto
       JOIN categoria c ON p.id_categoria = c.id_categoria
       JOIN pedido pe ON d.id_pedido = pe.id_pedido
WHERE  pe.fecha >= CURRENT_DATE - INTERVAL '3 months'
GROUP BY c.nombre
HAVING SUM(d.cantidad * d.precio_unitario) > 0
ORDER BY ingreso_total DESC;
```

6. Entregable de la Sesión 3

Al cierre de la sesión cada equipo entrega:

1. **Script DML de poblamiento** (02_dm1.sql): inserciones ejecutables inmediatamente después del DDL. Si usaron un script generador, incluirlo también.
2. **Script de programación almacenada** (03_plsql.sql): triggers, procedimientos y funciones, cada uno documentado con la regla de negocio que implementa.
3. **Script de consultas** (04_consultas.sql): las 15+ consultas organizadas por nivel, cada una con su pregunta de negocio como comentario.

4. **Volcado de la base de datos** (formato del SGBD): respaldo completo que permita restaurar la BD con estructura y datos.

Regla de reproducibilidad: la secuencia `01_ddl.sql` → `02_dml.sql` → `03_plsql.sql` → `04_consultas.sql` debe ejecutarse de principio a fin sin errores en una instancia limpia. Si algún script falla, el entregable se considera incompleto.

7. Cómo se evalúa esta sesión

El entregable de Sesión 3 aporta a tres criterios del proyecto:

encab		
Criterio: Consultas SQL y complejidad (20 pts)		
Cobertura de niveles N1–N4	8	Presencia y corrección de consultas en cada nivel de complejidad.
filaz Funciones de ventana (N5)	4	Uso correcto y pertinente de funciones analíticas.
Vistas (N6)	3	Vistas bien definidas que simplifiquen consultas frecuentes.
filaz Pertinencia de las preguntas de negocio	5	Las consultas responden preguntas relevantes del dominio, no ejercicios artificiales.
Criterio: Integridad y restricciones (3 pts restantes de Sesión 2)		
Triggers	2	Reglas de negocio correctamente implementadas y demostradas con casos de prueba.
filaz Procedimientos y funciones	1	Lógica de negocio encapsulada, con cursores y excepciones donde corresponde.
Criterio: Poblamiento, calidad y documentación (10 pts) — avance parcial		
Volumen y realismo de datos	4	Cumple mínimos de volumen. Datos realistas y coherentes.
filaz Cobertura de casos límite	2	Los datos incluyen escenarios que prueban restricciones y reglas de negocio.
Documentación de scripts	2	Comentarios claros en cada script. Estructura organizada y reproducible.

Los 2 puntos restantes de *Poblamiento, calidad y documentación* se evaluarán con la documentación final en Sesión 4.

8. Errores frecuentes a evitar

- **Datos de prueba triviales:** “test1”, “abc”, fechas todas iguales. Esto impide probar consultas de agregación y rangos temporales.
- **Consultas sin pregunta de negocio:** una consulta que no responde una pregunta real del dominio no demuestra capacidad analítica.
- **Todas las consultas del mismo nivel:** 15 consultas con `SELECT ... WHERE` sin JOINs ni agregación no cumplen el requisito de complejidad creciente.
- **Triggers que repiten restricciones declarativas:** un trigger que valide `NOT NULL` es redundante. Los triggers deben implementar lógica que **no puede** expresarse con DDL.
- **Procedimientos sin lógica real:** un procedimiento que solo hace un `INSERT` simple no encapsula lógica de negocio.
- **Funciones de ventana usadas sin contexto:** un `ROW_NUMBER()` sin partición ni propósito analítico no demuestra comprensión.
- **Scripts con dependencias de orden no documentadas:** si el trigger debe crearse antes de ejecutar el DML, eso debe ser explícito.

9. Recomendaciones

- **Piensen las consultas antes de poblar.** Definan las 15 preguntas de negocio primero; luego asegúrense de que los datos insertados permitan responderlas con resultados significativos.
- **Distribuyan el trabajo.** Mientras unos generan datos, otros pueden escribir triggers y otros pueden comenzar las consultas de nivel N1–N2 que solo necesitan la estructura.
- **Numeren sus archivos.** La convención `01_ddl.sql`, `02_dml.sql`, `03_plsql.sql`, `04_consultas.sql` facilita la ejecución secuencial y la revisión.
- **Prueben cada trigger con un caso positivo y uno negativo.** Documenten ambos casos como comentarios al final del trigger.
- **No dejen las funciones de ventana para el final.** Son el nivel más diferenciador y el que más puntos aporta proporcionalmente. Practiquen con sus datos reales.
- **Las vistas deben simplificar, no complicar.** Una buena vista encapsula un JOIN complejo que se reutiliza en varias consultas.

Este documento corresponde a la Sesión 3. Las instrucciones de las sesiones 4 y 5 se entregarán en su momento.